

Chapter 3 The Relational Model

Chapter Objectives

In this chapter you will learn:

- The origins of the relational model.
- The terminology of the relational model.
- How tables are used to represent data.
- The connection between mathematical relations and relations in the relational model.
- Properties of database relations.
- How to identify candidate, primary, alternate, and foreign keys.
- The meaning of entity integrity and referential integrity.
- The purpose and advantages of views in relational systems.

The Relational Database Management System (RDBMS) has become the dominant data-processing software in the world today, with estimated global software revenues of approximately US\$24 billion in 2011 and projected growth to roughly US\$37 billion by 2016. RDBMSs represent the second generation of DBMSs and are grounded in the relational data model proposed by E. F. Codd (1970). In the relational model, all data is logically organized within relations — tables. Each relation has a name and consists of named attributes (columns). Each tuple (row) holds one value per attribute. One of the greatest strengths of the relational model is this straightforward logical structure. Yet underlying that simplicity is a robust theoretical foundation that the first-generation DBMSs — network and hierarchical systems — conspicuously lacked.

Given the importance of RDBMSs, a significant portion of this book is devoted to them. This chapter covers the terminology and basic structural concepts of the relational data model. The next chapter examines the relational languages used for data retrieval and update.

3.1 Brief History of the Relational Model

The relational model was first articulated by E. F. Codd in his landmark paper "A relational model of data for large shared data banks" (Codd, 1970). This paper is now universally recognized as a milestone in the history of database systems, though a set-oriented model had been proposed somewhat earlier (Childs, 1968). The objectives Codd set out for the relational model were:

- To achieve a high degree of data independence, ensuring that application programs would not be affected by modifications to the internal data representation — particularly changes to file organizations, record orderings, or access paths.
 - To provide a sound basis for addressing data semantics, consistency, and redundancy.
- Codd's paper introduced the concept of normalized relations — relations that contain no

repeating groups.

- To enable the development of set-oriented data manipulation languages.

The most significant research following Codd's paper can be attributed to three projects, each approaching the relational model from a somewhat different angle.

The first was the System R project at IBM's San José Research Laboratory in California, developed during the late 1970s (Astrahan et al., 1976). System R was designed to demonstrate the practical viability of the relational model by implementing its data structures and operations. In doing so, it became a rich source of knowledge on implementation matters including transaction management, concurrency control, recovery, query optimization, security, integrity, human factors, and user interfaces. It generated a substantial body of research and spawned further prototypes. Most notably, System R produced two enduring outcomes:

- The development of SQL — Structured Query Language — which subsequently became the formal ISO and de facto standard language for relational DBMSs.
- A family of commercial relational DBMS products during the late 1970s and 1980s, including DB2 and SQL/DS from IBM and Oracle from Oracle Corporation.

The second significant project was INGRES (Interactive Graphics Retrieval System), conducted at the University of California at Berkeley at roughly the same time as System R. INGRES developed a prototype RDBMS pursuing research goals broadly similar to those of System R. The academic INGRES system helped spread understanding and appreciation of relational concepts, and gave rise to the commercial INGRES product from Relational Technology Inc. (now Actian Corporation) and the Intelligent Database Machine from Britton Lee Inc.

The third project was the Peterlee Relational Test Vehicle, conducted at the IBM UK Scientific Centre in Peterlee (Todd, 1976). More theoretically oriented than System R or INGRES, this project made notable contributions to the understanding of query processing and optimization, and to the functional extension of the relational model.

Commercial relational systems began appearing in the late 1970s and early 1980s. Today there are several hundred RDBMS products for both mainframe and personal computer environments, though many do not strictly conform to the formal definition of the relational model. PC-based examples include Office Access and Visual FoxPro from Microsoft, InterBase from Embarcadero Technologies, and R:Base from R:Base Technologies.

The influence of the relational model has extended beyond purely relational systems: many non-relational products now provide a relational interface, regardless of their underlying data model. IDMS, the primary network DBMS, evolved into CA-IDMS from CA Inc. (formerly Computer Associates) and now supports a relational view of data. Other mainframe DBMSs that incorporate relational features include Model 204 from Computer Corporation of America (a Rocket Software Inc. subsidiary) and ADABAS from Software AG.

Various extensions to the relational model have also been proposed, aimed at:

- More faithfully capturing the meaning and semantics of data (Codd, 1979).
 - Supporting object-oriented concepts (Stonebraker and Rowe, 1986).
 - Providing deductive capabilities (Gardarin and Valduriez, 1989).
-

3.2 Terminology

The relational model is built on the mathematical concept of a relation, which is physically represented as a table. Codd — a trained mathematician — drew his terminology from mathematics, principally set theory and predicate logic. This section explains the key terms and structural concepts of the relational model.

3.2.1 Relational Data Structure

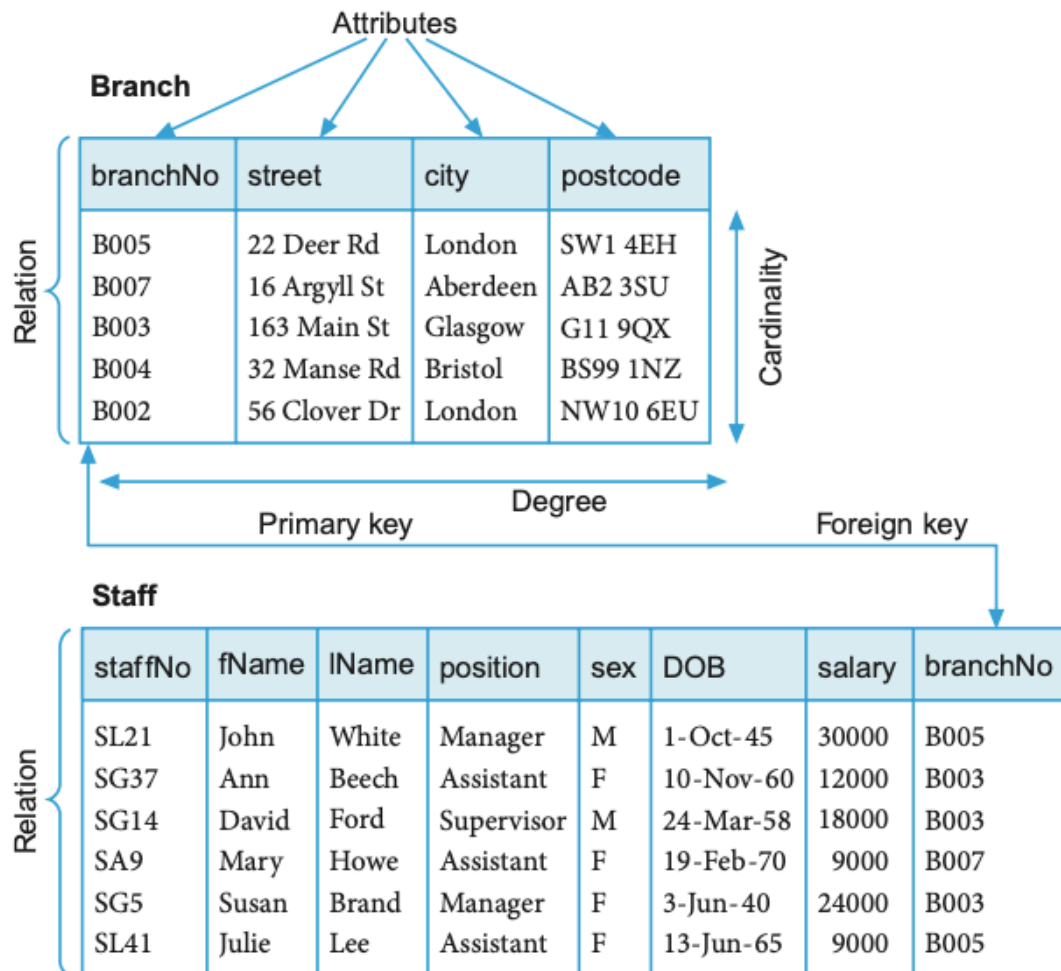
Relation: A table with columns and rows.

An RDBMS requires only that the user perceive the database as a collection of tables. It is important to note that this perception applies only to the logical structure of the database — the external and conceptual levels of the ANSI-SPARC architecture. It does not prescribe the physical storage implementation, which can employ a wide variety of storage structures.

Attribute: A named column of a relation.

Relations hold information about the objects to be represented in the database. A relation is depicted as a two-dimensional table where rows represent individual records and columns represent attributes. The order in which attributes appear is immaterial — the same relation and the same meaning are conveyed regardless of column ordering.

For example, information about branch offices is held in the Branch relation, with attributes branchNo, street, city, and postcode. Staff information is held in the Staff relation, with attributes staffNo, fName, lName, position, sex, DOB (date of birth), salary, and branchNo (the branch at which the staff member works). Figure 3.1 shows instances of both relations. As the figure illustrates, each column contains values drawn from a single attribute — the branchNo column, for instance, contains only branch office identifiers.



Domain: The set of allowable values for one or more attributes.

Domains are one of the most powerful features of the relational model. Every attribute is defined on a domain. Different attributes may share the same domain, or each may have its own. Figure 3.2 shows the domains for several attributes of the Branch and Staff relations. At any given time, many values that belong to a domain may not currently appear in the corresponding attribute column.

Attribute	Domain Name	Meaning	Domain Definition
branchNo	BranchNumbers	The set of all possible branch numbers	character: size 4, range B001–B999
street	StreetNames	The set of all street names in Britain	character: size 25
city	CityNames	The set of all city names in Britain	character: size 15
postcode	Postcodes	The set of all postcodes in Britain	character: size 8
sex	Sex	The sex of a person	character: size 1, value M or F
DOB	DatesOfBirth	Possible values of staff birth dates	date, range from 1-Jan-20, format dd-mmm-yy
salary	Salaries	Possible values of staff salaries	monetary: 7 digits, range 6000.00–40000.00

The domain concept is important because it allows the meaning and permissible range of attribute values to be defined in a single central location. This gives the system richer context when evaluating relational operations and enables it to prevent semantically meaningless comparisons. For example, comparing a street name with a telephone number makes no logical sense even though both might be represented as character strings at the implementation level. Conversely, multiplying a monetary value (monthly rental) by an integer (number of months on lease) is a semantically valid operation even though the operands belong to different domains. The complete implementation of domain support is non-trivial, and as a result many RDBMSs do not fully support it.

Tuple: A row of a relation.

The elements of a relation are its rows, or tuples. In the Branch relation, each row contains four values — one per attribute. The order of tuples within a relation is immaterial; rearranging the rows produces the same relation with the same meaning.

The structure of a relation — together with its domain specifications and any constraints on permissible values — is called the intension of the relation. The intension is generally stable, changing only when the definition of the relation itself is extended or modified. The collection of tuples at any given moment is called the extension (or state) of the relation, and this changes over time as data is inserted, updated, or deleted.

Degree: The degree of a relation is the number of attributes it contains.

Let's assume that we have a Branch relation which has four attributes, giving it degree four. Each row is thus a four-tuple. A relation with one attribute is called unary; one with two attributes is binary; one with three is ternary; beyond that, the term n-ary is used. Degree is a property of the relation's intension.

Cardinality: The cardinality of a relation is the number of tuples it contains.

Cardinality changes as tuples are added or deleted. Unlike degree, cardinality is a property of the extension and varies from one instance of the relation to the next.

Relational database: A collection of normalized relations with distinct relation names.

A relational database is composed of relations that are appropriately structured — a property referred to as normalization.

Alternative terminology

The terminology of the relational model can be confusing, partly because multiple sets of terms are in common use. Beyond the formal mathematical terms introduced above, a second alternative set exists, and a third set drawn from file-processing concepts is also encountered (reflecting the fact that a DBMS may physically store each relation as a file). The following table summarizes these alternatives.

FORMAL TERMS	ALTERNATIVE 1	ALTERNATIVE 2
Relation	Table	File
Tuple	Row	Record
Attribute	Column	Field

3.2.2 Mathematical Relations

To understand the true meaning of the term "relation," it is necessary to revisit some foundational concepts from mathematics. Given two sets $D_1 = \{2, 4\}$ and $D_2 = \{1, 3, 5\}$, their Cartesian product $D_1 \times D_2$ is the set of all ordered pairs in which the first element is a member of D_1 and the second is a member of D_2 :

$$D_1 \times D_2 = \{(2,1), (2,3), (2,5), (4,1), (4,3), (4,5)\}$$

Any subset of this Cartesian product is a relation. For example:

$$R = \{(2,1), (4,1)\}$$

This relation can also be expressed by stating the condition that selects it — in this case, all pairs whose second element is 1:

$$R = \{(x, y) \mid x \in D_1, y \in D_2, \text{ and } y = 1\}$$

Using the same sets, another relation S can be formed by selecting pairs where the first element is twice the second:

$$S = \{(x, y) \mid x \in D_1, y \in D_2, \text{ and } x = 2y\}$$

which yields:

$$S = \{(2,1)\}$$

since only the pair $(2,1)$ in the Cartesian product satisfies that condition. This concept extends naturally to three or more sets. For sets $D_1 = \{1,3\}$, $D_2 = \{2,4\}$, $D_3 = \{5,6\}$:

$$D_1 \times D_2 \times D_3 = \{(1,2,5), (1,2,6), (1,4,5), (1,4,6), (3,2,5), (3,2,6), (3,4,5), (3,4,6)\}$$

Any subset of these ordered triples is a relation. Generalizing to n sets $D_1, D_2, \dots, D_n$, their Cartesian product is:

$$D_1 \times D_2 \times \dots \times D_n = \{(d_1, d_2, \dots, d_n) \mid d_1 \in D_1, d_2 \in D_2, \dots, d_n \in D_n\}$$

Any set of n -tuples drawn from this Cartesian product constitutes a relation on those n domains. Crucially, defining a relation requires specifying the domains — the sets — from which values may be drawn.

3.2.3 Database Relations

Applying these mathematical concepts to the database context, we can define a relation schema.

Relation schema: A named relation defined by a set of attribute and domain name pairs.

Given attributes $A_1, A_2, \dots, A_n$ with corresponding domains $D_1, D_2, \dots, D_n$, the set $\{A_1:D_1, A_2:D_2, \dots, A_n:D_n\}$ is a relation schema. A relation R defined on a schema S is a set of mappings from attribute names to values drawn from their corresponding domains. Specifically, R is a set of n -tuples:

$(A_1:d_1, A_2:d_2, \dots, A_n:d_n)$ such that $d_1 \in D_1, d_2 \in D_2, \dots, d_n \in D_n$

In practice, when writing a relation as a table, we list attribute names as column headings and express each tuple as a row $(d_1, d_2, \dots, d_n)$ where each value is drawn from the appropriate domain. A table is simply a physical representation of a relation conceived as a subset of the Cartesian product of its attribute domains.

For the Branch relation in Figure 4.1, with attributes branchNo, street, city, and postcode, one tuple is:

$\{(\text{branchNo: B005, street: 22 Deer Rd, city: London, postcode: SW1 4EH})\}$

This is a relation instance. The Branch table is a convenient way of displaying all four-tuples that constitute the relation at a particular moment in time — which is why table rows in the relational model are called tuples.

Just as a relation has a schema, so does a relational database.

Relational database schema: A set of relation schemas, each with a distinct name.

If $R_1, R_2, \dots, R_n$ are a set of relation schemas, the relational database schema R is written as:

$R = \{R_1, R_2, \dots, R_n\}$

3.2.4 Properties of Relations

A relation has the following defining properties:

- The relation has a name that is unique within the relational schema.
- Each cell of the relation contains exactly one atomic (indivisible) value.
- Each attribute has a distinct name.
- The values within any given attribute column are all drawn from the same domain.
- No two tuples are identical — there are no duplicate rows.
- The ordering of attributes is immaterial.
- The ordering of tuples is theoretically immaterial, though in practice the physical order may influence access efficiency.

To see what these properties mean concretely, consider the Branch relation in Figure 4.1. Because each cell must hold exactly one value, storing two postcodes for a single branch in a single cell is prohibited. Relations contain no repeating groups. A relation satisfying this property is said to be normalized, or in first normal form. (Normal forms are discussed in Chapters 14 and 15.)

The column headings identify the attributes. All values in the branchNo column must come from the BranchNumbers domain — a postcode value, for instance, may not appear there. There can be no duplicate tuples; the row (B005, 22 Deer Rd, London, SW1 4EH) appears exactly once.

Columns can be reordered without changing the relation, provided attribute names travel with their values. Tuples can likewise be reordered: swapping the rows for branches B005 and B004 produces the same relation.

Most of these properties follow directly from the properties of mathematical relations:

- When forming the Cartesian product of simple single-valued sets, each element in each tuple is single-valued. Relations in the relational model share this property — each cell holds exactly one value. Note, however, that mathematical relations need not be normalized; Codd explicitly chose to disallow repeating groups in order to simplify the model.
- The permissible values in any position are constrained by the domain on which that position is defined.
- Because a mathematical set contains no repeated elements, a relation contains no duplicate tuples.
- Because sets are unordered, the order of tuples in a relation is immaterial.

One important difference from mathematical relations: in a standard ordered pair or n-tuple, position matters — (1,2) is distinct from (2,1). In the relational model, however, attribute order is explicitly declared immaterial, because column headings identify which attribute each value belongs to. The ordering of column headings in the intension does not matter; but once the structure is fixed, the values within each tuple must correspond to the attribute names in the order they are stored.

3.2.5 Relational Keys

Since no two tuples in a relation may be identical, it is always possible — and necessary — to identify one or more attributes, known as relational keys, that uniquely identify each tuple. This section explains the terminology for relational keys.

Superkey: An attribute, or set of attributes, that uniquely identifies a tuple within a relation.

A superkey guarantees unique identification of each tuple. However, a superkey may include attributes that are not strictly necessary for uniqueness. We are interested in identifying superkeys that are minimal — containing only the attributes needed for unique identification.

Candidate key: A superkey such that no proper subset is also a superkey within the relation.

A candidate key K for a relation R has two essential properties:

- **Uniqueness:** In every tuple of R , the values of K uniquely identify that tuple.
- **Irreducibility:** No proper subset of K possesses the uniqueness property.

A relation may have more than one candidate key. When a key consists of multiple attributes, it is called a composite key. Let's assume that we have a Branch relation where the attribute city cannot serve as a candidate key because multiple branch offices exist in London. However, branchNo uniquely identifies each branch and is therefore a candidate key. Similarly, postcode is also a candidate key for Branch.

Now consider a Viewing relation recording clients' inspections of properties, comprising clientNo, propertyNo, viewDate, and comment. Neither clientNo alone nor propertyNo alone is sufficient for unique identification, since a client may view multiple properties and a property may be viewed by multiple clients. The combination (clientNo, propertyNo) identifies at most one tuple, forming a composite candidate key for Viewing. If a client could view the same property more than once, viewDate would need to be added to the composite key; for simplicity, we assume this is not the case.

An important point: the data present in a relation at any given moment cannot prove that an attribute combination is a candidate key. The absence of duplicates in a particular instance is no guarantee that duplicates are impossible. However, the presence of duplicates in an instance can definitively prove that a combination is not a candidate key. Identifying a candidate key requires understanding the real-world semantics of the data — only semantic knowledge can establish with certainty whether duplicates are possible. For example, in the last Branch relation, the surname "White" appears only once in the Staff relation, but a future staff member with the same surname would invalidate IName as a candidate key.

Primary key: The candidate key selected to identify tuples uniquely within the relation.

Since a relation cannot contain duplicate tuples, every relation has at least one candidate key and therefore always has a primary key. In the worst case, all attributes together form the primary key, but typically a smaller subset suffices. Candidate keys that are not chosen as the primary key are called alternate keys. For Branch, if branchNo is selected as the

primary key, postcode becomes an alternate key. For Viewing, the only candidate key is the composite (clientNo, propertyNo), which automatically becomes the primary key.

Foreign key: An attribute, or set of attributes, within one relation that matches the candidate key of some (possibly the same) relation.

When an attribute appears in more than one relation, it typically represents a relationship between tuples of those relations. The attribute branchNo appears in both the Branch and Staff relations. In Branch, it is the primary key. In Staff, it serves as a link connecting each staff member to the branch where they work — making it a foreign key in Staff that targets the primary key branchNo in Branch. These shared attributes play a central role in data manipulation, as the next chapter demonstrates.

3.2.6 Representing Relational Database Schemas

A relational database consists of any number of normalized relations. The relational schema for part of the DreamHome case study is as follows (with primary keys underlined by convention):

Branch	(<u>branchNo</u> , street, city, postcode)
Staff	(<u>staffNo</u> , fName, lName, position, sex, DOB, salary, branchNo)
PropertyForRent	(<u>propertyNo</u> , street, city, postcode, type, rooms, rent, ownerNo, staffNo, branchNo)
Client	(<u>clientNo</u> , fName, lName, telNo, prefType, maxRent, eMail)
PrivateOwner	(<u>ownerNo</u> , fName, lName, address, telNo, eMail, password)
Viewing	(<u>clientNo</u> , <u>propertyNo</u> , viewDate, comment)
Registration	(<u>clientNo</u> , <u>branchNo</u> , staffNo, dateJoined)

The standard convention is to write the relation name followed by its attribute names in parentheses, with the primary key underlined. The conceptual schema of the database is the complete collection of all such relation schemas. Figure 4.3 shows an instance of this relational schema.

3.3 Integrity Constraints

The previous section addressed the structural component of the relational data model. A data model also has a manipulative component — defining what operations are permitted on the data — and a set of integrity constraints that ensure data accuracy. This section discusses the relational integrity constraints; the next chapter covers relational manipulation operations.

An example of an integrity constraint was already introduced in Section 2.2.1: because every attribute is defined on a domain, domain constraints restrict the set of permissible values for each attribute. Beyond domain constraints, there are two particularly important integrity rules — constraints that apply to all instances of the database. These are known as entity integrity and referential integrity. Other types of integrity constraint include multiplicity and general constraints (introduced in Section 2.3.4). Before defining entity and referential integrity, we must first clarify the concept of nulls.

3.3.1 Nulls

Null: Represents a value for an attribute that is currently unknown or is not applicable for this tuple.

A null can be thought of as carrying the logical meaning "unknown." It may signify that a value is not applicable to a particular tuple, or simply that no value has yet been provided. Nulls offer a mechanism for handling incomplete or exceptional data.

It is critical to understand that a null is not the same as the numeric value zero or a string of blank spaces. Zero and blank spaces are actual values; a null represents the complete absence of a value. Consequently, nulls must be handled differently from other values. Some authors use the phrase "null value," but since a null is not a value — it represents the absence of one — this expression is discouraged.

Branch

branchNo	street	city	postcode
B005	22 Deer Rd	London	SW1 4EH
B007	16 Argyll St	Aberdeen	AB2 3SU
B003	163 Main St	Glasgow	G11 9QX
B004	32 Manse Rd	Bristol	BS99 1NZ
B002	56 Clover Dr	London	NW10 6EU

Staff

staffNo	fName	lName	position	sex	DOB	salary	branchNo
SL21	John	White	Manager	M	1-Oct-45	30000	B005
SG37	Ann	Beech	Assistant	F	10-Nov-60	12000	B003
SG14	David	Ford	Supervisor	M	24-Mar-58	18000	B003
SA9	Mary	Howe	Assistant	F	19-Feb-70	9000	B007
SG5	Susan	Brand	Manager	F	3-Jun-40	24000	B003
SL41	Julie	Lee	Assistant	F	13-Jun-65	9000	B005

PropertyForRent

propertyNo	street	city	postcode	type	rooms	rent	ownerNo	staffNo	branchNo
PA14	16 Holhead	Aberdeen	AB7 5SU	House	6	650	CO46	SA9	B007
PL94	6 Argyll St	London	NW2	Flat	4	400	CO87	SL41	B005
PG4	6 Lawrence St	Glasgow	G11 9QX	Flat	3	350	CO40		B003
PG36	2 Manor Rd	Glasgow	G32 4QX	Flat	3	375	CO93	SG37	B003
PG21	18 Dale Rd	Glasgow	G12	House	5	600	CO87	SG37	B003
PG16	5 Novar Dr	Glasgow	G12 9AX	Flat	4	450	CO93	SG14	B003

Client

clientNo	fName	lName	telNo	prefType	maxRent	eMail
CR76	John	Kay	0207-774-5632	Flat	425	john.kay@gmail.com
CR56	Aline	Stewart	0141-848-1825	Flat	350	astewart@hotmail.com
CR74	Mike	Ritchie	01475-392178	House	750	mr Ritchie01@yahoo.co.uk
CR62	Mary	Tregear	01224-196720	Flat	600	maryt@hotmail.co.uk

PrivateOwner

ownerNo	fName	lName	address	telNo	eMail	password
CO46	Joe	Keogh	2 Fergus Dr, Aberdeen AB2 7SX	01224-861212	jkeogh@lhh.com	*****
CO87	Carol	Farrel	6 Achray St, Glasgow G32 9DX	0141-357-7419	cfarrel@gmail.com	*****
CO40	Tina	Murphy	63 Well St, Glasgow G42	0141-943-1728	tinam@hotmail.com	*****
CO93	Tony	Shaw	12 Park Pl, Glasgow G4 0QR	0141-225-7025	tony.shaw@ark.com	*****

Viewing

clientNo	propertyNo	viewDate	comment
CR56	PA14	24-May-13	too small
CR76	PG4	20-Apr-13	too remote
CR56	PG4	26-May-13	
CR62	PA14	14-May-13	no dining room
CR56	PG36	28-Apr-13	

Registration

clientNo	branchNo	staffNo	dateJoined
CR76	B005	SL41	2-Jan-13
CR56	B003	SG37	11-Apr-12
CR74	B003	SG37	16-Nov-11
CR62	B007	SA9	7-Mar-12

In the Viewing relation of the example above, for instance, the comment attribute may be undefined until a prospective tenant has visited a property and submitted feedback. Without null support, the only alternatives are to insert artificial placeholder values (such as -1) or to add extra attributes (such as a hasCommentBeenSupplied flag), both of which can be confusing and misleading to users.

Nulls introduce implementation difficulties arising from the fact that the relational model is grounded in first-order predicate calculus — a two-valued (Boolean) logic in which the only permitted values are true and false. Incorporating nulls requires reasoning in a three- or four-valued logic (Codd, 1986, 1987, 1990), which substantially complicates evaluation and comparison operations.

The inclusion of nulls in the relational model remains a contentious topic. Codd ultimately embraced nulls as an integral feature (Codd, 1990). Critics argue that the underlying problem of missing information is not yet sufficiently well understood, that no entirely satisfactory solution has emerged, and that incorporating nulls into the relational model is therefore premature (see, for example, Date, 1995).

3.3.2 Entity Integrity

The first integrity rule governs the primary keys of base relations. For now, we define a base relation as a relation corresponding to an entity in the conceptual schema.

Entity integrity: In a base relation, no attribute of a primary key can be null.

By definition, a primary key is a minimal identifier used to distinguish tuples uniquely. This means that no proper subset of a primary key is sufficient for unique identification on its own. Allowing any component of a primary key to be null would imply that not all its attributes are required for identification — directly contradicting the definition of a primary key.

For example, since branchNo is the primary key of the Branch relation, no tuple may be inserted into Branch with a null value for branchNo. Similarly, for the Viewing relation — whose composite primary key consists of clientNo and propertyNo — no tuple may be inserted with a null for clientNo, a null for propertyNo, or nulls for both.

Examining this rule carefully reveals some subtleties. First, why does the rule apply specifically to primary keys and not to all candidate keys, which are also unique identifiers? Second, why is it restricted to base relations? Consider a query that extracts the comment attribute from the Viewing relation. The resulting unary relation has comment as its primary key, yet this attribute contains nulls in Figure 4.3. Because this result is not a base relation, the model permits the primary key to be null. Various attempts to reformulate this rule more precisely have been made (see, for example, Codd, 1988 and Date, 1990).

3.3.3 Referential Integrity

The second integrity rule governs foreign keys.

Referential integrity: If a foreign key exists in a relation, either the foreign key value must match a candidate key value of some tuple in its home relation, or the foreign key value must be wholly null.

For example, branchNo in the Staff relation is a foreign key targeting branchNo in the Branch relation (the home relation). It must not be possible to create a staff record referencing branch number B025 unless a record for B025 already exists in the Branch relation. However, it should be possible to create a staff record with a null branch number — for instance, to accommodate a newly hired employee who has not yet been assigned to a specific branch.

3.3.4 General Constraints

General constraints: Additional rules specified by users or database administrators that define or constrain some aspect of the enterprise.

Users may specify additional constraints beyond entity and referential integrity that the data must satisfy. For example, if a branch office may have at most 20 staff members, a constraint to that effect should be expressible, and the DBMS should enforce it by rejecting any attempt to add a twenty-first staff member to that branch. In practice, the level of support for general constraints varies considerably from one RDBMS to another.

3.4 Views

In the three-level ANSI-SPARC architecture, an external view was described as the structure of the database as it appears to a particular user. In the relational model, the term "view" carries a somewhat more specific meaning. Rather than representing a user's complete external model, a view is a virtual or derived relation — a relation that does not necessarily exist in its own right but can be dynamically derived from one or more base relations on demand. An external model may therefore consist of both base relations at the conceptual level and views derived from those base relations. This section provides a brief introduction to views in relational systems. A more detailed treatment — including how views are created and used in SQL.

3.4.1 Terminology

The relations discussed throughout this chapter up to this point are known as base relations.

Base relation: A named relation corresponding to an entity in the conceptual schema, whose tuples are physically stored in the database.

Views are defined in terms of base relations.

View: The dynamic result of one or more relational operations applied to base relations, producing another relation. A view is a virtual relation that does not necessarily exist in the database but can be produced on request, at the time of the request.

A view appears to the user as though it were a fully realized relation and can be queried and manipulated as if it were a base relation. However, unlike base relations, a view's tuples are not physically stored — only the view's definition is stored, in the system catalog. The contents of a view are defined by a query over one or more base relations. Any operation performed on the view is automatically translated by the DBMS into equivalent operations on the underlying base relations. Views are dynamic: changes to the base relations that affect a view are immediately reflected in the view, and changes that users make to a view (where permitted) are propagated back to the underlying base relations. This section describes the purpose of views and summarizes the restrictions that apply to view updates..

3.4.2 Purpose of Views

The view mechanism is valuable for several reasons:

- It provides a powerful and flexible security mechanism by hiding portions of the database from users who should not see them. A user who accesses data through a view has no awareness of attributes or tuples that are excluded from that view.
- It allows different users to see the same underlying data in different ways simultaneously, each tailored to their individual needs and familiarity with the data.
- It can simplify complex operations on base relations. For example, if a view is defined as a join of two relations (see Section 5.1), users can issue simpler queries against the view, which the DBMS will automatically translate into the corresponding join operations.

A view should be designed to match the external model that the user naturally works with. Concrete examples include:

- A user might need Branch tuples that also include the name of the branch manager. This view would be created by combining the Branch relation with a restricted projection of the Staff relation containing only staff with the position "Manager."
 - Some staff members should see Staff tuples that exclude the salary attribute.
 - Attributes may be renamed or reordered to match a user's preference — for instance, a user accustomed to the full name "Branch Number" rather than the attribute identifier branchNo can have the column heading displayed accordingly.
 - Certain staff members should be able to see only the property records for properties they personally manage.

All of these examples illustrate the view mechanism as a source of logical data independence. Beyond this, views also support a more significant form of logical data independence that accommodates the reorganization of the conceptual schema. If a new attribute is added to a relation, existing users whose views do not include that attribute remain completely unaware of it. If an existing relation is restructured or split, a view can be defined so that those users continue to interact with the data exactly as before.

3.4.3 Updating Views

Any update made to a base relation should be immediately reflected in all views derived from that relation. Conversely, when a user modifies a view, the change should be applied to the underlying base relation. However, there are constraints on which types of modification are permissible through a view. The following conditions summarize when most systems allow updates through a view:

- Updates are permitted through views defined by a simple query on a single base relation that includes the primary key or a candidate key of that relation.
- Updates are not permitted through views that involve multiple base relations.
- Updates are not permitted through views that involve aggregation or grouping operations.

Views can be classified as theoretically non-updatable, theoretically updatable, or partially updatable. A comprehensive survey of the issues surrounding the updating of relational views can be found in Furtado and Casanova (1985).

Chapter Summary

- The RDBMS has become the dominant data-processing software in the world, representing the second generation of DBMSs and based on the relational data model proposed by E. F. Codd.
- A mathematical relation is a subset of the Cartesian product of two or more sets. In database terms, a relation is any subset of the Cartesian product of the domains of its attributes, normally written as a set of n-tuples in which each element is drawn from the appropriate domain.
- Relations are physically represented as tables, with rows corresponding to individual tuples and columns corresponding to attributes.
- The structure of a relation — together with domain specifications and other constraints — constitutes part of the intension of the database. The relation with all its current tuples written out is an instance, or extension, of the database.
- The defining properties of database relations are: each cell contains exactly one atomic value; attribute names are distinct; attribute values within a column are drawn from the same

domain; attribute ordering is immaterial; tuple ordering is immaterial; and no two tuples are identical.

- The degree of a relation is the number of attributes it contains; the cardinality is the number of tuples. A relation with one attribute is unary, two is binary, three is ternary, and the general term for n attributes is n -ary.

- A superkey is an attribute or set of attributes that uniquely identifies tuples in a relation. A candidate key is a minimal superkey. A primary key is the candidate key selected to identify tuples; every relation must have a primary key. A foreign key is an attribute or set of attributes in one relation that corresponds to a candidate key in another relation.

- A null represents a value for an attribute that is currently unknown or is not applicable for a particular tuple.

- Entity integrity states that no attribute of a primary key in a base relation may be null. Referential integrity states that a foreign key value must either match a candidate key value in some tuple of the home relation or be wholly null. Beyond these two principal integrity rules, other integrity constraints include domain constraints, required data constraints, multiplicity constraints, and general constraints.

- A view in the relational model is a virtual or derived relation, dynamically constructed from the underlying base relations when needed. Views provide security and allow the designer to customize how data is presented to different users. Not all views are updatable.